

SQLAlchemy 비동기 문법

SQLAlchemy 비동기 문법

- 비동기적인 방식으로 SQLAlchemy를 FastAPI에서 사용하기 위해, SQLAlchemy의 비동기 지원 버전을 사용해야 함
- 이를 위해 sqlalchemy.ext.asyncio 모듈을 활용할 것임
- 비동기 방식은 I/O 작업이 많은 웹 어플리케이션에 응답성을 향상시키는데 유용함

Sqlalchemy.ext.asyncio

- 코드에서 사용된 sqlalchemy.ext.asyncio 모듈은 SQLAlchemy1.4 이상에서 도입된 공식 비동기 지원을 위한 모듈
- 이 모듈은 비동기 데이터베이스 엔진(create_async_engine), 비동기 세션(AsyncSession), 그리고 비동기 세션팩토리(sessionmaker에 class_=AsyncSession 옵션 사용)를 포함
- 이러한 구성 요소를 사용하여 SQLAlchemy의 ORM 기능을 비동기적으로 사용할 수 있음

Sqlalchemy.ext.asyncio

- 데이터베이스 모델 정의(비동기 방식)

- aiomysql은 Python의 비동기 I/O 지원을 위한 asyncio와 MySQL 사이의 비동기적인 어댑터
- FastAPI 어플리케이션에서 비동기 SQLAlchemy를 사용하고, MySQL 데이터베이스와 비동기적으로 통신하기 위해 필요

- `pip install sqlalchemy==2.0.27`
- `pip install aiomysql==0.2.0`

Sqlalchemy.ext.asyncio

- 비동기 데이터베이스 연결 설정

```
from sqlalchemy.ext.asyncio import create_async_engine

# 비동기 데이터베이스 설정을 위한 문자열을 정의.
# 이 문자열에는 사용자 이름, 비밀번호, 서버 주소, 데이터베이스 이름이 포함.
DATABASE_URL = "mysql+pymysql://username:password@localhost/fastapi"

# SQLAlchemy의 비동기 엔진을 생성.
engine = create_async_engine(DATABASE_URL, echo=True)
```

- create_async_engine을 사용해 비동기 엔진 생성
- 이는 비동기 데이터베이스 드라이버(aiomysql)를 사용하여 비동기적으로 데이터베이스 작업 수행
- echo=True 옵션은 실행되는 모든 SQL 쿼리가, 콘솔이나 로그에 출력하라는 옵션으로 디폴트는 False

Sqlalchemy.ext.asyncio

- 세션 생성

```
from sqlalchemy.orm import sessionmaker

# 비동기 세션 생성을 위한 세션메이커를 정의.
AsyncSessionLocal = sessionmaker(
    autocommit=False, autoflush=False, bind=engine, class_=AsyncSession
)
```

- sessionmaker 함수에 class_=AsyncSession 인자를 전달하여 비동기 세션 팩토리를 생성
- 이를 통해 생성된 세션은 비동기적 데이터베이스 작업을 지원함

Sqlalchemy.ext.asyncio

- 데이터베이스 작업 수행

```
# 비동기 데이터베이스 세션을 생성하고 관리하는 의존성 함수를 정의.  
async def get_db():  
    async with AsyncSessionLocal() as session:  
        yield session  
        await session.commit()  
)
```

- 비동기 방식에서는 async와 await 키워드를 사용하여 비동기 작업 수행
- async def get_db()는 비동기 함수를 선언하며, 이 함수 내부에서 비동기 I/O 작업을 처리할 수 있음
- async with 구문은 비동기 컨텍스트 매니저를 사용하여 세션의 생성과 소멸을 비동기적으로 관리
- await session.commit()은 세션의 변경사항을 데이터베이스에 비동기적으로 커밋함

Sqlalchemy.ext.asyncio

- 데이터베이스 테이블 생성

```
@asynccontextmanager
async def app_lifespan(app: FastAPI):
    # 애플리케이션 시작 시 실행될 로직
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield
    # 애플리케이션 종료 시 실행될 로직 (필요한 경우)

# FastAPI 애플리케이션을 초기화.
app = FastAPI(lifespan=app_lifespan)
```

- @asynccontextmanager

- Python의 contextlib 모듈에서 제공하는 데코레이터. 이 데코레이터는 비동기 컨텍스트 매니저를 쉽게 생성할 수 있게 해주는 기능을 제공
- 비동기 컨텍스트 매니저는 async with 문을 사용할 때 실행 컨텍스트를 관리하는 객체. 이는 리소스의 할당과 해체를 비동기적으로 관리하는 데 유용
- @asynccontextmanager를 사용하여 정의된 함수는 async with 문 내에서 사용할 수 있으며, yield 전에 실행되는 초기화 코드와 yield 후에 실행되는 종료 코드를 포함할 수 있음

Sqlalchemy.ext.asyncio

- 데이터베이스 테이블 생성

- `async def app_lifespan(app: FastAPI):`

- 이 함수는 비동기 컨텍스트 매니저로서 FastAPI 어플리케이션의 생명주기 동안 특정 작업을 수행함
 - `app:FastAPI` 파라미터는 이 컨텍스트 매니저가 관리할 FastAPI 어플리케이션 인스턴스를 나타냄

- `async with engine.begin() as conn:`

- `async with` 문은 비동기 컨텍스트 매니저를 사용하여 리소스의 비동기적 할당과 해제를 관리함
 - `engine.begin()`은 SQLAlchemy의 비동기 엔진에서 트랜잭션을 시작하는 비동기 컨텍스트 매니저. 이는 데이터베이스 작업을 트랜잭션 단위로 관리하려는 경우 사용
 - `as conn`은 `engine.begin()`에 의해 생성된 비동기 연결 객체를 `conn` 변수에 할당함

- `await conn.run_sync(Base.metadata.create_all)`

- `await` 키워드는 비동기 작업이 완료될 때까지 기다림.
 - `conn.run_sync()` 메서드는 비동기 연결 객체에서 동기 함수를 비동기적으로 실행할 수 있게 해주는 메서드. 이를 통해 동기적인 SQLAlchemy 메서드를 비동기적 컨텍스트 내에서 사용할 수 있음
 - `Base.metadata.create_all`은 SQLAlchemy의 Declarative Base를 사용하여 정의된 모든 테이블을 데이터베이스에서 생성하는 메서드. 만약 테이블이 이미 존재한다면, 이 작업은 무시

Sqlalchemy.ext.asyncio

- 데이터베이스 테이블 생성

- yield
 - 함수 실행을 일시 중지시키고, 함수 호출자에게 제어를 반환함
 - 이 코드에서는 어플리케이션이 실행중인 동안 일시 중지됨
 - 어플리케이션의 시작 로직과 종료 로직 사이의 분기점 역할을 함
- `app = FastAPI(lifespan=app_lifespan)`
 - FastAPI 어플리케이션 인스턴스를 생성하면서, `lifespan` 이벤트 처리를 위한 비동기 컨텍스트 매니저를 등록함
 - `lifespan=app_lifespan`는 어플리케이션의 시작과 종료 시점에 `app_lifespan` 함수에 정의된 로직을 실행하도록 FastAPI에 지시함

- 주의 사항

- 비동기 세션 관리에서는 `async with` 구문을 사용하여 세션의 생성과 소멸을 자동으로 처리함. 이는 비동기 작업에서 자주 사용되는 패턴으로, 코드의 복잡성을 줄이고 자원 관리를 효율적으로 할 수 있게 도와 줌
- 비동기 방식에서는 세션 커밋도 비동기적으로 처리해야 함. 예제에서는 `yield session` 후에 `await session.commit()`를 호출하여 변경 사항을 데이터베이스에 비동기적으로 커밋함